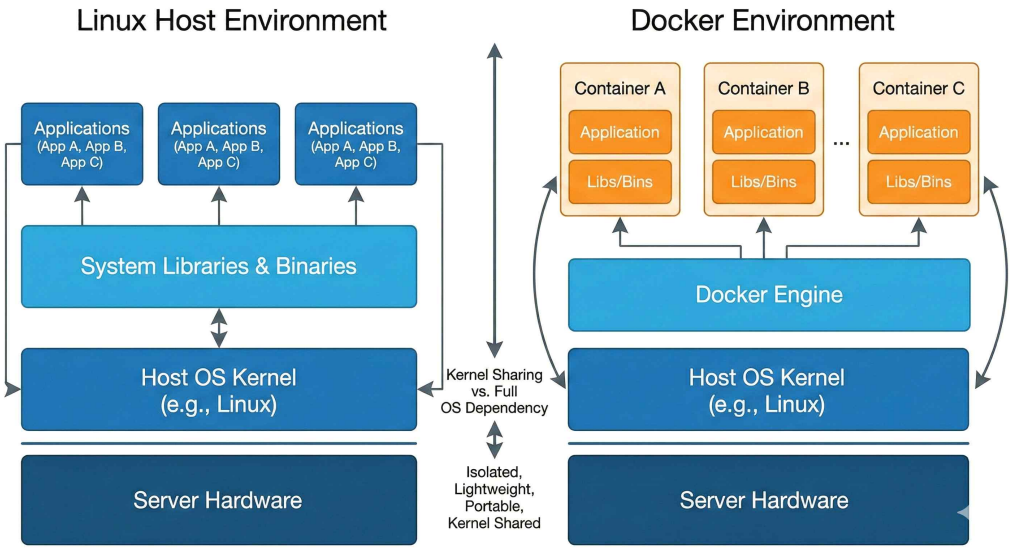
****** host와 docker 환경 차이 **********



********docker 설치 가이드*********

<도커 설치>

1. 패키지 업데이트

sudo apt-get update

2. 필수 패키지 설치

sudo apt-get install -y ca-certificates curl gnupg lsb-release

3. 도커 GPG 키 추가

curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg || { echo "GPG key download failed"; exit 1; }

4. 도커 저장소 설정

echo "deb [arch=amd64 signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

5. 도커 설치

sudo apt-get update

sudo apt-get install docker-ce docker-ce-cli containerd.io 도커 엔진 설치하기

docker --version 도커버전 확인

sudo docker run hello-world 헬로월드 이미지 실행 확인 (테스트)

sudo apt-get install docker-compose-plugin 도커 컴포즈 플러그인 설치

// 기타 참고 (옵션)

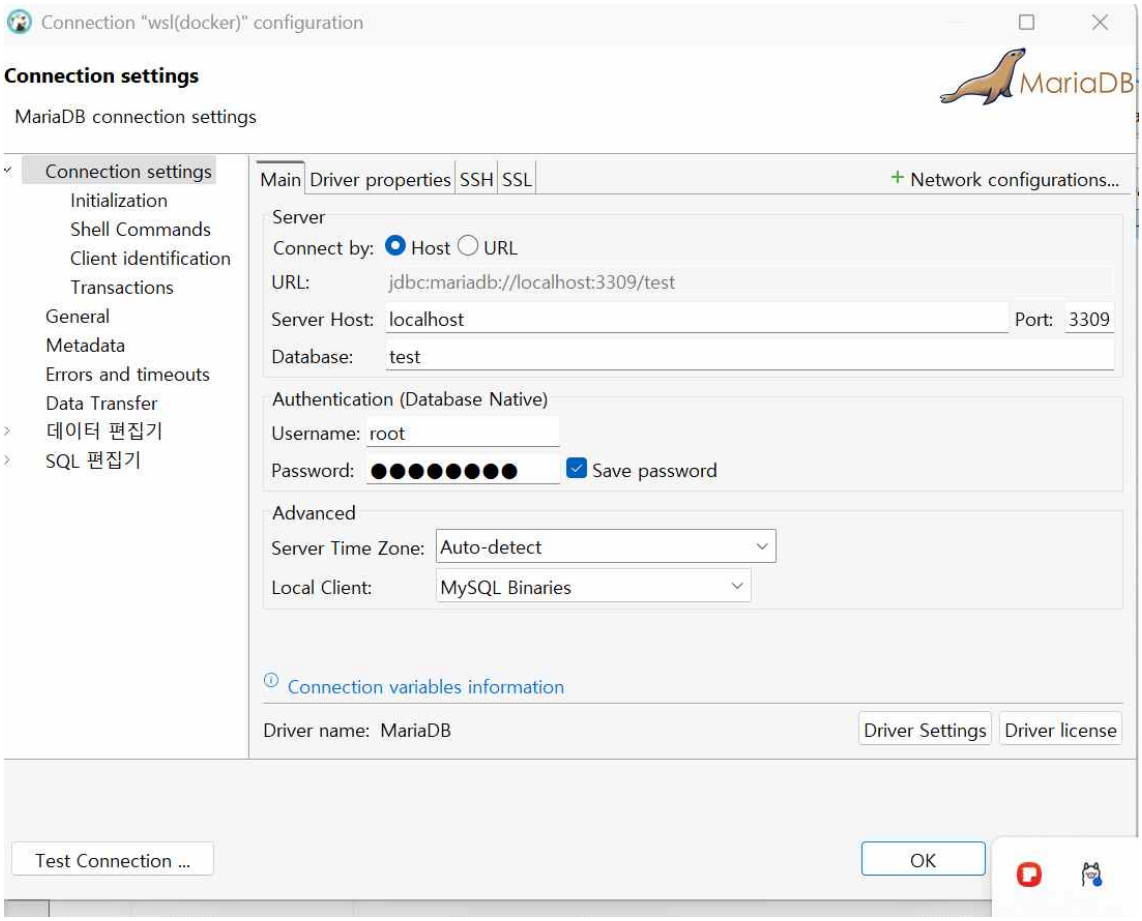
whami 또는 echo $USER 접속중인 사용자

sudo usermod -aG docker 사용자명 sudo 없이 사용 ---> logout 이후 사용 가능

<docker mariadb 설치 및 접속>

sudo docker pull mariadb:10.6 // mariadb 이미지 설치

sudo docker run --name mariadb -p 3309:3306 -e MYSQL_ROOT_PASSWORD=1234 -d mariadb:10.6 //컨테이너 생성
sudo docker exec -it mariadb bash //mariadb 접속
mariadb -u root -p
create database test; // 데이터베이스 생성
show databases; 데이터베이스 조회
dbeaver 접속 확인



application.properties 포트 확인 (웹서버 포트, mariadb포트)

```
spring.application.name=springChatbot

spring.datasource.driver-class-name=org.mariadb.jdbc.Driver
# mariadb local setting
# spring.datasource.url=jdbc:mariadb://localhost:3306/test?useUnicode=yes&characterEncoding=UTF-8&
# mariadb server host setting
# spring.datasource.url=jdbc:mariadb://localhost:3308/test?useUnicode=yes&characterEncoding=UTF-8&
# mariadb docker setting
spring.datasource.url=jdbc:mariadb://mariadb:3309/test?useUnicode=yes&characterEncoding=UTF-8&allo

spring.datasource.username=root
spring.datasource.password=8948864a
server.port=8080
```

ChatService.java 에서 fastapi.url 확인 =>hostname -I에서 서버 IP주소로 입력

```java
package springChatbot.service;

import springChatbot.dto.ChatRequest;

@Service
public class ChatService {

    // @Value("${fastapi.url:http://localhost:8000}")
    @Value("${fastapi.url:http://172.19.198.186:8000}")
    private String fastApiUrl;

    private final RestTemplate restTemplate;
    private final ChatHistoryMapper chatHistoryMapper;

    @Autowired
    public ChatService(ChatHistoryMapper chatHistoryMapper) {
        this.restTemplate = new RestTemplate();
        this.chatHistoryMapper = chatHistoryMapper;
    }
```

<tomcat 설치>

sudo docker pull tomcat:9.0

sudo docker run -d -it --name tomcat9 -p 8081:8080 tomcat:9.0 //Tomcat 컨테이너 생성

sudo docker exec -it tomcat9 /bin/bash // Tomcat 컨테이너 접속

```
hgahn@ads:~$ sudo docker ps -a
CONTAINER ID   IMAGE                COMMAND                  CREATED        STATUS                      PORTS
                                      NAMES
cf3946c122a4   tomcat:9.0           "catalina.sh run"        2 months ago   Up 10 minutes               0.0.0.0:8081->8080/tcp
:8081->8080/tcp                       tomcat99
3ce3622ae32e   mariadb:10.6         "docker-entrypoint.s…"   2 months ago   Up 8 minutes                0.0.0.0:3308->3306/tcp
:3308->3306/tcp                       mariadb
dd99493ad03f   tomcat:9.0           "catalina.sh run"        2 months ago   Exited (143) 2 months ago
                                      tomcat9
df378ec972f2   jenkins/jenkins:lts  "/usr/bin/tini -- /u…"   2 months ago   Exited (255) 2 months ago   8080/tcp, 50000/tcp, 0
:8085->8085/tcp, [::]:8085->8085/tcp   jenkins
hgahn@ads:~$ sudo docker exec -it tomcat99 /bin/bash
root@cf3946c122a4:/usr/local/tomcat# ls
bin           conf            filtered-KEYS  LICENSE  native-jni-lib  README.md      RUNNING.txt  upstream-KEYS  webapps.dist
BUILDING.txt  CONTRIBUTING.md  lib            logs     NOTICE          RELEASE-NOTES  temp         webapps        work
root@cf3946c122a4:/usr/local/tomcat# cd conf
root@cf3946c122a4:/usr/local/tomcat/conf# ls
Catalina         catalina.properties  jaspic-providers.xml  logging.properties  tomcat-users.xml  web.xml
catalina.policy  context.xml          jaspic-providers.xsd  server.xml          tomcat-users.xsd
root@cf3946c122a4:/usr/local/tomcat/conf# vi server.xml
```

/// 기타 tomcat 환경설정 ///

// conf/server.xml (connector port="해당포트", appBase="webapps.dist“)

// conf/tomcat-users.xml (rolename, username, password 추가)

// conf/Catalina/localhost/manager.xml 아래내용 추가 (X)

<Context privileged="true" antiResourceLocking="false" docBase="{catalina.home}/webapps.dist/manager">

<!--<valve className="org.apach.catalina.valves.RemoteAddrValue" allow="127\.\d+\.\d+\.\d+|::1|0:0:0:0:0:0:0:1|172.17.0.4" />-->

</Context>

// Tomcat 사용자 설정 확인/수정

$TOMCAT_HOME/conf/tomcat-users.xml 파일에 다음 내용 추가

<tomcat-users>

  <role rolename="manager-gui"/>

  <role rolename="manager-script"/>

  <role rolename="manager-jmx"/>

  <role rolename="manager-status"/>

```
  <user username="admin" password="1234"
    roles="manager-gui,manager-script,manager-jmx,manager-status"/>
</tomcat-users>
```

```
<!--
  <role rolename="tomcat"/>
  <role rolename="role1"/>
  <user username="tomcat" password="<must-be-changed>" roles="tomcat"/>
  <user username="both" password="<must-be-changed>" roles="tomcat,role1"/>
  <user username="role1" password="<must-be-changed>" roles="role1"/>
-->
  <role rolename="manager-gui"/>
  <role rolename="manager-script"/>
  <role rolename="manager-jmx"/>
  <role rolename="manager-status"/>
  <user username="admin" password="1234"
    roles="manager-gui,manager-script,manager-jmx,manager-status"/>
</tomcat-users>
"tomcat-users.xml" 62L, 3003B
```

// Tomcat의 Context 설정 추가

$TOMCAT_HOME/webapps.dist/manager/META-INF/context.xml 파일 수정

```
<Context antiResourceLocking="false" privileged="true" >
  <Valve className="org.apache.catalina.valves.RemoteAddrValve"
         allow=".*" />
</Context>
```

```
<Context antiResourceLocking="false" privileged="true" >
  <CookieProcessor className="org.apache.tomcat.util.http.Rfc6265CookieProcessor"
                   sameSiteCookies="strict" />
  <!-- <Valve className="org.apache.catalina.valves.RemoteAddrValve"
  allow="127\.\d+\.\d+\.\d+|::1|0:0:0:0:0:0:0:1" /> -->
  <Valve className="org.apache.catalina.valves.RemoteAddrValve"
         allow=".*" />
  <Manager sessionAttributeValueClassNameFilter="java\.lang\.(?:Boolean|Integer|Long|Number|String)|org\.apa
srfPreventionFilter\$LruCache(?:\$1)?|java\.util\.(?:Linked)?HashMap"/>
</Context>
```

<컨테이너 간의 네트워크 연결 및 확인>

Docker 네트워크를 생성하고 mariadb, tomcat 컨테이너들을 연결

docker network create my-network

docker network connect my-network tomcat-container 이름 --> sudo docker network connect my-network tomcat9.0

docker network connect my-network mariadb-container 이름 --> sudo docker network connect my-network mariadb

docker network connect my-network jenkins-container 이름 --> sudo docker network connect my-network jenkins

네트워크 연결 상태 확인

sudo docker network inspect my-network

application.properties에서 수정(가상 mariadb 컨테이너 ip보다는 컨테이너 명으로 변경)

spring.datasource.driver-class-name=org.mariadb.jdbc.Driver
spring.datasource.url=jdbc:mariadb://**mariadb**:3306/survey?useUnicode=yes&characterEncoding=UTF-8&allowMultiQueries=true&serverTimezone=Asia/Seoul

<JENKINS 설치 -- docker기반>
sudo docker pull jenkins/jenkins:lts-jdk17
sudo docker run -itd --name jenkins --env JENKINS_OPTS=--httpPort=8085 -p 8085:8085 jenkins/jenkins:lts-jdk17 컨테이너 생성(lts 버전, 8080 포트변경)
sudo docker exec -it jenkins /bin/bash cf. 관리자권한 접속 시 exec -itu 0로 접속
cat /var/jenkins_home/secrets/initialAdminPassword jenkins 패스워드 확인

<docker 기본 명령어>
docker 로그 : sudo docker logs [컨테이너명]
--> ex. sudo docker logs -f --tail 100 <컨테이너명> // 100줄까지 로그 확인
sudo chmod 777 -R /var/lib/docker/containers/* 컨테이너 하위 폴더 권한 부여
find . -name "*tar*" 현재 디렉토리에서 tar가 포함된 파일 찾기
sudo docker rmi $(docker images -q) 도커이미지 한번에 지우기
sudo docker images 이미지 조회
sudo docker ps -a 컨테이너 목록 출력
sudo docker start '컨테이너ID'
sudo docker exec -it '컨테이너ID' /bin/bash
apt-get update ex. 톰캣 컨테이너 폴더에서 vim 설치 이전 업데이트
apt-get install vim ex. 톰캣 컨테이너 폴더에서 vim 설치
sudo docker commit mysql(컨테이너) mysql_image(이미지명) 컨테이너를 이미지로 저장
mysql -u root -p MySQL서버 접속
sudo docker cp ROOT.war 컨테이너명:/usr/local/tomcat/webapps.dist (ex. sudo docker cp ROOT.war tomcat9:/usr/local/tomcat/webapps.dist)
sudo docker cp mysql-connector-j-8.0.31.jar 톰캣컨테이너명:/usr/local/tomcat/lib (ex. sudo docker cp mysql-connector-j-8.0.31.jar tomcat9:/usr/local/tomcat/lib)
sudo docker cp /source/* container_name:/target/ Host-->Container Copy
sudo docker cp container_name:/source/* /target/ Container --> Host Copy
sudo docker pull mysql
sudo docker run --name mysql -e MYSQL_ROOT_PASSWORD=8948864a -d -p 3306:3306 mysql:8.0.31 MySQL 컨테이너 생성

<도커엔진 삭제>
sudo apt-get purge -y docker-engine docker docker.io docker-ce docker-ce-cli
sudo apt-get autoremove -y --purge docker-engine docker docker.io docker-ce

<도커컨테이너, 이미지, 볼륨, 사용자가 만든 파일 전부 삭제>
sudo rm -rf /var/lib/docker /etc/docker
sudo rm /etc/apparmor.d/docker
sudo groupdel docker
sudo rm -rf /var/run/docker.sock

<docker mysql 자동실행(부팅시)>
docker에서 생성된 mysql 컨테이너 부팅시 자동 실행 (데몬에 서비스 생성 방법)
/etc/systemd/system에 서비스 작성
sudo vi docker_mysql.service

[Unit]
Wants=docker_mysql.service
After=docker_mysql.service
[Service]
RemainAfterExit=yes
ExecStart=/usr/bin/docker start mysql(mysql 컨테이너명)
ExecStop=/usr/bin/docker stop mysql(mysql 컨테이너명)
[Install]
WantedBy=multi-user.target
---
systemctl daemon-reload // 데몬 재시작
systemctl enable docker_mysql.service // mysql 서비스 등록
systemctl status docker_mysql.service // 서비스 확인
systemctl list-units --type=service // 모든 서비스 상태 표시
service --status-all|grep + // 현재 실행중인 서비스 확인
docker 컨테이너 업데이트 명령어 사용하는 방법
sudo docker update --restart=always <컨테이너id> 자동실행
sudo docker update --restart=no <컨테이너id> 자동실행 취소

<dockerfile + docker compose 예시>
현재 디렉토리에서 Dockerfile 빌드
docker build -t my-jenkins:lts . // my-jenkins라는 이미지 생성, (.)은 현재 디렉토리
빌드된 이미지 확인
docker images

// jenkins 도커파일
FROM jenkins/jenkins:lts

Jenkins HTTP 포트를 8085로 설정
ENV JENKINS_OPTS="--httpPort=8085"

```
# 컨테이너 내부에서 8085 포트 노출
EXPOSE 8085

# Jenkins 기본 사용자로 실행
USER jenkins
```

//docker compose

```
version: '3.8'

services:
  mariadb:
    image: mariadb:10.6
    container_name: mariadb
    ports:
      - "3309:3306"
    environment:
      - MYSQL_ROOT_PASSWORD=1234
    networks:
      - my-network
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 10s
      timeout: 5s
      retries: 3

  tomcat:
    image: tomcat:9.0
    container_name: tomcat9
    ports:
      - "8081:8080"
    networks:
      - my-network
    depends_on:
      mariadb:
        condition: service_healthy

  jenkins:
    build:
      context: .
      dockerfile: Dockerfile
    image: my-jenkins:lts
    container_name: jenkins
    ports:
      - "8085:8085"
    networks:
      - my-network
    volumes:
      - jenkins_home:/var/jenkins_home
    depends_on:
      - tomcat
```

```
networks:
  my-network:
    name: my-network
    driver: bridge

volumes:
  jenkins_home:
```

<dockerfile없이 docker compose 하나로 생성 예시>

docker-compose up -d // docker compse 실행

docker-compose stop //서비스 중지

docker-compose down // 서비스 중지 및 컨테이너 제거

dokcer-compmose logs // 로그 확인

// docker-compose.yml

```
version: '3.8'

services:
  mariadb:
    image: mariadb:10.6
    container_name: mariadb
    ports:
      - "3309:3306"
    environment:
      - MYSQL_ROOT_PASSWORD=1234
    networks:
      - my-network
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 10s
      timeout: 5s
      retries: 3

  tomcat:
    image: tomcat:9.0
    container_name: tomcat9
    ports:
      - "8081:8080"
    networks:
      - my-network
    depends_on:
      mariadb:
        condition: service_healthy

  jenkins:
    image: jenkins/jenkins:lts
    container_name: jenkins
    ports:
```

```yaml
      - "8085:8085"
    environment:
      - JENKINS_OPTS=--httpPort=8085
    networks:
      - my-network
    volumes:
      - jenkins_home:/var/jenkins_home
    depends_on:
      - tomcat

networks:
  my-network:
    name: my-network
    driver: bridge

volumes:
  jenkins_home:
```